Doc. no. P0007R0
Date: 2015-09-28
Project: Programming Language C++
Reply to: ADAM David Alan Martin (<adamartin@FreeBSD.org>)
Alisdair Meredith (<ameredith1@bloomberg.net>)

Constant View: A proposal for a `std::as_const` helper function template

Version History

- N4380 Original submission
- P0007R0 Added proposed wording (this paper)

This paper was presented to LEWG at the Lenexa meeting in May 2015, and greeted favorably. It was held back from moving to LWG only for the lack of proposed wording, added in this update.

I. Table of Contents

II. Introduction

This paper proposes a new helper function template `std::as_const`, which would live in the `<utility>` header. A simple example usage:

```
#include <utility>
#include <type_traits>

void
demoUsage()
{
    std::string mutableString= "Hello World!";
    const std::string &constView= std::as_const( mutableString );

    assert( &constView == mutableString );
    assert( &std::as_const( mutableString ) == mutableString );

    using WhatTypeIsIt= std::remove_reference_t< decltype( std::as_const( mutableString ) >;

    static_assert( std::is_same< std::remove_const_t< WhatTypeIsIt >, std::string >::value,
            "WhatTypeIsIt should be some kind of string." );
    static_assert( !std::is_same< WhatTypeIsIt, std::string >::value,
            "WhatTypeIsIt shouldn't be a mutable string." );
}
```

III. Background

The C++ Language distinguishes between 'const Type' and 'Type' in ADL lookup for selecting function overloads. The selection of overloads can occur among functions like:

```
int processEmployees( std::vector< Employee > &employeeList );
bool processEmployees( const std::vector< Employee > &employeeList );
```

Oftentimes these functions should have the same behavior, but sometimes free (or member) functions will return different types, depending upon which qualifier (`const` or non-`const`) applies to the source type. For example, `std::vector< T >::begin` has two overloads, one returning `std::vector< T >::iterator`, and the other returning `std::vector< T >::const_iterator`. For this reason `cbegin` was added to the container member-function manifest.

IV. Motivation

A larger project often needs to call functions, like `processEmployees`, and selecting among specific `const` or non-`const` overloads. Further, within C++11 and newer contexts, passing an object for binding or perfect forwarding can often require specifying a `const` qualifier applies to the actual object. This can also be useful in specifying that a template be instantiated as adapting to a "`const`" reference to an object, instead of a non-`const` reference.

V. Alternatives

1. Continue use the hard-to-use idiom `const_cast< const T & >( object )`
2. Use a more modern idiom `const_cast< std::add_const< decltype( object ) >::type & >( object )`
3. Provide a language extension, i.e.: `const_cast( object )` which is equivalent to the above
4. Provide a modified cast form, i.e.: `(const) object` which is equivalent to the above
5. Provide a new trailing cast-like form, i.e.: `object const`, or `object const.blah` or `object.blah const`
6. Provide a library function, `as_const`, which this paper proposes

VI. Discussion

This conversion, or alternative-viewpoint, is always safe. Following the rule that safe and common actions shouldn't be ugly while dangerous and infrequent actions should be hard to write, we can conclude that this addition of `const` is an operation that should not be ugly and hard. Const-cast syntax is ugly and therefore its usage is inherently eschewed.

Regarding the above alternatives, each can be discussed in turn:

`const_cast< const T & >( object )`:

The benefits of this form are:

- It is portable to older compilers (e.g. C++98/03 compilers)
- It is widely understood
- It is not misleading

The drawbacks of this form are:

- It is a significant amount of typing for a relatively simple operation
- It requires the programmer to have knowledge of `decltype( object )`
- It is phrased in a known-dangerous cast form (`const_cast`)

`const_cast< std::add_const< decltype( object ) >::type & >( object )`:

The benefits of this form are:

- It is supported by C++11 and beyond
- It requires no modifications to the C++ standard
- It does not require the programmer to have knowledge of `decltype( object )`; it merely requires that he use that type-deduction

The drawbacks of this form are:

- It is not supported by C++98/03
- It is an even more burdensome amount of typing
- It is phrased in an incredibly ugly and hard-to-understand form
- It is phrased using a form which requires an extra `typename` usage, when used in a template context, due to the `add_const` metafunction
- It is phrased in a known-dangerous cast form (`const_cast`)

Extend the language with `const_cast( object )`:

The benefits of this form are:

- It is a simple, straightforward, and understandable construct
- It uses a presently ill-defined form

The drawbacks of this form are:

- It is not supported by anything, at present
- It provides an alternate, potentially confusing difference for `const_cast`, over the other `_cast` forms currently in the core language
- It requires a core language extension for only marginal gain
- It is phrased in a known-dangerous cast form, which is locatable by regex

Extend the language with `(const) object`:

The benefits of this form are:

- It is a simple, straightforward, and understandable construct
- It uses a presently ill-defined form
- It is extremely terse

The drawbacks of this form are:

- It is not supported by anything, at present
- It requires a core language extension for only marginal gain
- It is phrased in the legacy C-style casting form, which is considered (by many) to be both outmoded and ugly

Library function, `std::as_const( object )`:

The benefits of this form are:

- It is simple, straightforward, and understandable construct
- It requires no new core language extensions
- It is implementable in C++98/03 terms, for those who wish to back-port the feature
- It is terse
- It is not misleading
- It does not require the programmer to know `decltype( object )`
- It follows the precedent set by `std::move` and `std::forward`, in terms of wrapping commonly needed, but difficult to write cast expressions

The drawbacks of this form are:

- It may not have the best name.
- Proposed names included:
  - `std::constify( object )`
  - `std::to_const( object )`
  - `std::constant( object )`
  - `std::view_const( object )`
  - `std::make_const( object )`
  - `std::add_const( object )`
  - `std::const_view( object )`

In proposing `std::as_const( object )`, we feel that the name is sufficiently terse yet descriptive. Every other name had at least one drawback.

VII. Proposed Implementation

In the `<utility>` header, the following code should work:

```
// ...
// -- Assuming that the file has reverted to the global namespace --
namespace std
{
    template< typename T >
```

```
    inline typename std::add_const< T >::type &
    as_const( T &t ) noexcept
    {
        return t;
    }

}
// ...
```

VIII. Proposed Wording

Insert the following signature into the `<utility>` synopsis, between 20.2.4 // foward/move, and 20.2.5 // declval:

```
template <class T> add_const_t<T>& as_const(T& t) noexcept;
```

Insert the following subclause between **20.2.4 forward/move helpers [forward]**, and **20.2.5 Function template declval [declval]**:

20.2.x `const`-casting utility [utility.const.cast]

```
template <class T> add_const_t<T>& as_const(T& t) noexcept;
```

1 *Returns:* `t`.

IX. Further Discussion

The above implementation only supports safely re-casting an l-value as `const` (even if it may have already been `const`). It is probably desirable to have xvalues and prvalues also be usable with `as_const`, but there are some issues to consider. Some examples of contexts we'd like to consider for an expanded proposal are:

```
std::string getString();
auto &x= as_const( getString() );
auto &x= const_cast< const std::string & >( getString() );


void function( std::string & );
void function( const std::string & );

function( as_const( getString() ) );
```

An alternative implementation which would support all of the forms used above, would be:

```
template< typename T >
inline const T &
as_const( const T& t ) noexcept
{
    return t;
}

template< typename T >
inline const T
as_const( T &&t ) noexcept( noexcept( T( t ) ) )
{
    return t;
}
```

We believe that such an implementation helps to deal with lifetime extension issues for temporaries which are captured by `as_const`, but we have not fully examined all of the implications of these forms. We are open to expanding the scope of this proposal, but we feel that the utility of a simple-to-use `as_const` is sufficient even without the expanded semantics.